DBMS PRACTICALS – COMPLETE SQL QUERY BOOK

MySQL / XAMPP Compatible | Database + Table Creation + Practical Queries

This document combines the practicals shown in the supplied syllabus image. Run the SQL in MySQL Workbench or phpMyAdmin (XAMPP). Use a test database because some commands delete or change data.

Contents

1. Practical 1: ER scenarios and relational tables
2. Practical 2: DDL – CREATE, ALTER, DROP
3. Practical 3: View databases/tables and update/delete records
4. Practical 4: DML – INSERT, UPDATE, DELETE
5. Practical 5: Simple queries, aggregates, GROUP BY, HAVING, date/string/math functions
6. Practical 6: INNER, OUTER and NATURAL JOIN
7. Practical 7: Search conditions, summary queries, subqueries, IN and EXISTS
8. Practical 8: Database/table space, users and roles
9. Practical 9: Backup/recovery, COMMIT, ROLLBACK, SAVEPOINT
10. Practical 10: VIEW and INDEX

Practical 1: ER Model and Conversion into Relational Tables

The syllabus asks for two scenarios. The following four scenarios are provided so you can choose any two. Primary keys are marked PK and foreign keys FK.

Scenario A – Online Banking System

- CUSTOMERS(customer_id PK, name, email)
- ACCOUNTS(account_id PK, customer_id FK, balance)
- TRANSACTIONS(transaction_id PK, account_id FK, amount, type, transaction_time)
- Relationship: One customer can own many accounts (1:M); one account can have many transactions (1:M).

```
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL
);
CREATE TABLE Accounts (
  account_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
CREATE TABLE Transactions (
  transaction_id INT PRIMARY KEY AUTO_INCREMENT,
  account_id INT NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  type ENUM('deposit','withdrawal','transfer') NOT NULL,
  transaction_time DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (account_id) REFERENCES Accounts(account_id)
);
```

Scenario B – E-Commerce Inventory

- CUSTOMERS(customer_id PK, name, email)

- PRODUCTS(product_id PK, name, price, stock)
- ORDERS(order_id PK, customer_id FK, order_date)
- ORDER_ITEMS(order_item_id PK, order_id FK, product_id FK, quantity)
- Relationships: Customer–Orders 1:M; Orders–Order_Items 1:M; Products–Order_Items 1:M.

```
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL
);
CREATE TABLE Products (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0
);
CREATE TABLE Orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
CREATE TABLE Order_Items (
  order_item_id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

Scenario C – Hospital Management System

- PATIENTS(patient_id PK, name, age, gender)
- DOCTORS(doctor_id PK, name, specialization)
- APPOINTMENTS(appointment_id PK, patient_id FK, doctor_id FK, appointment_date)
- TREATMENTS(treatment_id PK, appointment_id FK, diagnosis, prescription)
- Relationships: Patient–Appointments 1:M; Doctor–Appointments 1:M; Appointment–Treatments 1:M.

```
CREATE TABLE Patients (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  age INT,
  gender VARCHAR(20)
);
CREATE TABLE Doctors (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  specialization VARCHAR(100)
);
CREATE TABLE Appointments (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  patient_id INT NOT NULL,
  doctor_id INT NOT NULL,
  appointment_date DATETIME NOT NULL,
  FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
  FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);
CREATE TABLE Treatments (
  treatment_id INT PRIMARY KEY AUTO_INCREMENT,
  appointment_id INT NOT NULL,
  diagnosis VARCHAR(255),
```

```
  prescription VARCHAR(255),
  FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

Scenario D – Library Management System

- BOOKS(book_id PK, title, author, copies_available)
- STUDENTS(student_id PK, name, department)
- ISSUES(issue_id PK, book_id FK, student_id FK, issue_date, return_date)
- Relationships: Book–Issues 1:M; Student–Issues 1:M.

```
CREATE TABLE Books (
  book_id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(150) NOT NULL,
  author VARCHAR(100),
  copies_available INT NOT NULL DEFAULT 0
);
CREATE TABLE Students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  department VARCHAR(100)
);
CREATE TABLE Issues (
  issue_id INT PRIMARY KEY AUTO_INCREMENT,
  book_id INT NOT NULL,
  student_id INT NOT NULL,
  issue_date DATE NOT NULL,
  return_date DATE NULL,
  FOREIGN KEY (book_id) REFERENCES Books(book_id),
  FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

Note: Customers is reused in Banking and E-Commerce, so do not create the same table twice in one database. Run only the scenario(s) you need, or use separate databases for each scenario.

Common Setup: Create Database and Sample Tables

For Practicals 2–10, use this compact college-store database. It has related tables and sample data for DDL, DML, functions, joins, subqueries, views and indexes.

```
CREATE DATABASE IF NOT EXISTS dbms_practical;
USE dbms_practical;

CREATE TABLE Departments (
  department_id INT PRIMARY KEY AUTO_INCREMENT,
  department_name VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE Students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  student_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) UNIQUE,
  department_id INT,
  admission_date DATE,
  marks DECIMAL(5,2) DEFAULT 0,
  city VARCHAR(60),
  FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(100) NOT NULL,
```

```
  fee DECIMAL(10,2) NOT NULL,
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  enrollment_date DATE,
  grade VARCHAR(5),
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

INSERT INTO Departments(department_name) VALUES
('Computer Engineering'), ('E&TC'), ('Mechanical');

INSERT INTO Students(student_name,email,department_id,admission_date,marks,city) VALUES
('Aarav Patil','aarav@example.com',1,'2024-07-10',85,'Pune'),
('Sara Shaikh','sara@example.com',1,'2024-07-12',92,'Solapur'),
('Rohan Jadhav','rohan@example.com',2,'2023-08-01',68,'Pune'),
('Neha More','neha@example.com',2,'2024-07-15',74,'Mumbai'),
('Imran Khan','imran@example.com',3,'2023-08-05',55,'Solapur');

INSERT INTO Courses(course_name,fee,department_id) VALUES
('Database Management',2500,1),
('Computer Networks',2200,1),
('Digital Electronics',2000,2),
('Embedded Systems',2800,2),
('Workshop Technology',1800,3);

INSERT INTO Enrollments(student_id,course_id,enrollment_date,grade) VALUES
(1,1,'2025-01-10','A'),
(1,2,'2025-01-11','B'),
(2,1,'2025-01-10','A+'),
(3,3,'2025-01-12','B'),
(4,4,'2025-01-13','A'),
(5,5,'2025-01-14','C');
```

Practical 2: DDL Commands – CREATE, ALTER, DROP

DDL (Data Definition Language) changes the structure of database objects.

```
-- CREATE TABLE
CREATE TABLE Demo (
  id INT PRIMARY KEY,
  name VARCHAR(50)
);

-- ALTER TABLE: add a column
ALTER TABLE Demo ADD email VARCHAR(100);

-- ALTER TABLE: modify a column
ALTER TABLE Demo MODIFY name VARCHAR(100);

-- ALTER TABLE: rename a column (MySQL 8.0+)
ALTER TABLE Demo RENAME COLUMN email TO student_email;

-- DROP TABLE (permanently removes table structure and data)
DROP TABLE Demo;
```

Practical 3: View Databases/Tables and Update/Delete Records

```
SHOW DATABASES;
USE dbms_practical;
SHOW TABLES;
DESCRIBE Students;
SHOW CREATE TABLE Students;

-- Display all records
SELECT * FROM Students;

-- Update a record (example)
UPDATE Students SET city='Nashik' WHERE student_id=3;

-- Delete a record (example; use WHERE carefully)
DELETE FROM Students WHERE student_id=5;
```

Always use a WHERE clause with UPDATE and DELETE when you intend to affect only selected records.

Practical 4: DML Commands – INSERT, UPDATE, DELETE

DML (Data Manipulation Language) is used to add, modify and remove table records.

```
-- INSERT
INSERT INTO Students(student_name,email,department_id,admission_date,marks,city)
VALUES ('Kiran Deshmukh','kiran@example.com',1,'2025-07-01',81,'Pune');

-- UPDATE
UPDATE Students SET marks=88 WHERE student_id=1;

-- DELETE
DELETE FROM Enrollments WHERE enrollment_id=6;
```

Practical 5: Simple Queries, Aggregate Functions, GROUP BY, HAVING, Date/String/Math Functions

```
-- Simple SELECT and WHERE
SELECT student_name, marks FROM Students;
SELECT * FROM Students WHERE marks >= 75;
SELECT * FROM Students WHERE city='Pune' AND marks > 70;
SELECT * FROM Students ORDER BY marks DESC;

-- Aggregate functions
SELECT COUNT(*) AS total_students FROM Students;
SELECT AVG(marks) AS average_marks FROM Students;
SELECT MAX(marks) AS highest_marks, MIN(marks) AS lowest_marks FROM Students;
SELECT SUM(fee) AS total_course_fees FROM Courses;

-- GROUP BY and HAVING
SELECT city, COUNT(*) AS student_count
FROM Students GROUP BY city;

SELECT department_id, AVG(marks) AS average_marks
FROM Students GROUP BY department_id
HAVING AVG(marks) >= 70;

-- Date functions
SELECT student_name, YEAR(admission_date) AS admission_year FROM Students;
SELECT student_name, DATE_FORMAT(admission_date,'%d-%m-%Y') AS formatted_date FROM Students;
SELECT CURDATE() AS today, NOW() AS current_datetime;
SELECT student_name, DATEDIFF(CURDATE(), admission_date) AS days_since_admission FROM Students;
```

```
-- String functions
SELECT UPPER(student_name), LOWER(city) FROM Students;
SELECT student_name, LENGTH(student_name) AS name_length FROM Students;
SELECT CONCAT(student_name, ' - ', city) AS student_details FROM Students;
SELECT SUBSTRING(student_name,1,3) AS short_name FROM Students;

-- Mathematical functions
SELECT ROUND(AVG(marks),2) AS rounded_average FROM Students;
SELECT ABS(-25) AS absolute_value, CEIL(12.2) AS ceiling_value, FLOOR(12.8) AS floor_value;
SELECT POWER(2,3) AS power_value, MOD(17,5) AS remainder;
```

Practical 6: Joins – INNER, OUTER, NATURAL JOIN

```
-- INNER JOIN: only matching records
SELECT s.student_name, d.department_name
FROM Students s
INNER JOIN Departments d ON s.department_id=d.department_id;

-- LEFT OUTER JOIN: all students, including those without a matching department
SELECT s.student_name, d.department_name
FROM Students s
LEFT JOIN Departments d ON s.department_id=d.department_id;

-- RIGHT OUTER JOIN: all departments, including those without students
SELECT s.student_name, d.department_name
FROM Students s
RIGHT JOIN Departments d ON s.department_id=d.department_id;

-- NATURAL JOIN: automatically joins columns with the same name
SELECT student_name, department_name
FROM Students NATURAL JOIN Departments;

-- Three-table join
SELECT s.student_name, c.course_name, e.grade
FROM Enrollments e
JOIN Students s ON e.student_id=s.student_id
JOIN Courses c ON e.course_id=c.course_id;
```

MySQL does not provide FULL OUTER JOIN directly. It can be emulated with LEFT JOIN UNION RIGHT JOIN if required.

Practical 7: Search Conditions, Summary Queries, Subqueries, IN and EXISTS

```
-- Search conditions
SELECT * FROM Students WHERE marks BETWEEN 60 AND 90;
SELECT * FROM Students WHERE city IN ('Pune','Solapur');
SELECT * FROM Students WHERE student_name LIKE 'S%';
SELECT * FROM Students WHERE email IS NOT NULL;

-- Summary query
SELECT department_id, COUNT(*) AS total, AVG(marks) AS avg_marks
FROM Students GROUP BY department_id;

-- Subquery: students scoring above overall average
SELECT student_name, marks
FROM Students
WHERE marks > (SELECT AVG(marks) FROM Students);

-- IN clause: students enrolled in a course
SELECT student_name FROM Students
WHERE student_id IN (SELECT student_id FROM Enrollments);
```

```
-- IN with a named course
SELECT student_name FROM Students
WHERE student_id IN (
  SELECT e.student_id FROM Enrollments e
  JOIN Courses c ON e.course_id=c.course_id
  WHERE c.course_name='Database Management'
);

-- EXISTS: students with at least one enrollment
SELECT s.student_name
FROM Students s
WHERE EXISTS (
  SELECT 1 FROM Enrollments e WHERE e.student_id=s.student_id
);

-- NOT EXISTS: students with no enrollment
SELECT s.student_name
FROM Students s
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollments e WHERE e.student_id=s.student_id
);
```

Practical 8: Database/Table Space, Users and Roles

In MySQL, “tablespace” management is more engine/server-specific. These are common database-level and user/role commands; run user/role commands with an account that has sufficient privileges.

```
-- Database creation and selection
CREATE DATABASE IF NOT EXISTS college_extra;
USE college_extra;

-- Create a user (replace password with a strong private password)
CREATE USER 'lab_user'@'localhost' IDENTIFIED BY 'ChangeThis_StrongPassword!';

-- Grant permissions
GRANT SELECT, INSERT, UPDATE ON dbms_practical.* TO 'lab_user'@'localhost';

-- Inspect grants
SHOW GRANTS FOR 'lab_user'@'localhost';

-- Revoke selected permissions
REVOKE INSERT, UPDATE ON dbms_practical.* FROM 'lab_user'@'localhost';

-- Create and assign a role (MySQL 8.0+)
CREATE ROLE 'student_readonly';
GRANT SELECT ON dbms_practical.* TO 'student_readonly';
GRANT 'student_readonly' TO 'lab_user'@'localhost';
SET DEFAULT ROLE 'student_readonly' TO 'lab_user'@'localhost';

-- Remove user when no longer needed
DROP USER 'lab_user'@'localhost';
```

Do not use a real personal password in a practical report. Some XAMPP/MySQL installations or hosting accounts may not grant permission to create users, roles or tablespaces.

Practical 9: Backup and Recovery, COMMIT, ROLLBACK, SAVEPOINT

Transaction examples work with transactional tables such as InnoDB. SQL statements below demonstrate transaction control; database backup itself is usually done with mysqldump or phpMyAdmin Export.

```
-- Transaction control example
START TRANSACTION;
UPDATE Students SET marks=marks+2 WHERE student_id=1;
SAVEPOINT before_second_change;
UPDATE Students SET city='Nagpur' WHERE student_id=1;

-- Undo only the change after the savepoint
ROLLBACK TO SAVEPOINT before_second_change;

-- Make the remaining change permanent
COMMIT;

-- Full rollback example
START TRANSACTION;
UPDATE Students SET marks=0 WHERE student_id=2;
ROLLBACK;

-- Backup from Windows command prompt (not inside SQL editor):
-- mysqldump -u root -p dbms_practical > dbms_practical_backup.sql

-- Restore from Windows command prompt:
-- mysql -u root -p dbms_practical < dbms_practical_backup.sql
```

In phpMyAdmin: select dbms_practical → Export → choose SQL → Go. To restore, select/create the target database → Import → choose the .sql file → Go.

Practical 10: Create VIEW and INDEX for Large Tables

```
-- Create a view
CREATE OR REPLACE VIEW StudentDepartmentView AS
SELECT s.student_id, s.student_name, s.email, s.marks, d.department_name
FROM Students s
LEFT JOIN Departments d ON s.department_id=d.department_id;

-- Query a view like a table
SELECT * FROM StudentDepartmentView;
SELECT * FROM StudentDepartmentView WHERE marks >= 80;

-- Create indexes
CREATE INDEX idx_students_name ON Students(student_name);
CREATE INDEX idx_students_city_marks ON Students(city, marks);
CREATE INDEX idx_enrollments_student ON Enrollments(student_id);

-- View indexes
SHOW INDEX FROM Students;

-- Remove an index if needed
DROP INDEX idx_students_name ON Students;

-- Remove a view if no longer needed
DROP VIEW IF EXISTS StudentDepartmentView;
```

Suggested Practical Record Format

- Aim: Write the aim of the practical from the syllabus.
- Theory: Add a short definition of the command or concept.
- SQL Queries: Copy the relevant query block and execute it.
- Output: Paste a screenshot of successful execution and result rows from MySQL/phpMyAdmin.
- Conclusion: State that the required SQL commands were executed and verified.

Important Execution Notes

- Run the Common Setup block once before Practicals 2–10.
- Run Practical 1 scenario scripts separately if table names overlap (for example, Customers).
- Use semicolons after SQL statements.
- If a table already exists, either skip its CREATE TABLE statement or use a fresh practice database.
- Be careful with DROP TABLE, DROP DATABASE, DELETE and UPDATE because they can remove or change data.
- For MySQL user/role commands, use an account with sufficient privileges; some shared or restricted installations block these operations.